

专篇讲解 · 不是总手册压缩版

## 每一步的目的、原理、 与上一步的衔接、怎么做

模块  $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H$

每一步固定四节，做完一节再翻下一节

本机：ROS 1 Noetic · 最内层 robot\_gripper

/home/biand/workspace\_ws/src/robot\_gripper\_9\_91/robot\_gripper\_5\_29/robot\_gripper

旁路包 vs\_practice · 调用 moveit\_commander · 不改 abb\_driver

目的：为什么做这一层。原理：在算什么、谁在哪一层。衔接：上一步交出了什么、这一步补哪一层。怎么做：本机命令与通过标准。

文件性质：专篇讲解（不是总手册的压缩版）

读法：每一步固定四节——目的、原理、与上一步的衔接、怎么做。做完一节再翻下一节。

本机：WSL Ubuntu 20.04 · ROS 1 Noetic · 工作空间根

/home/biand/workspace\_ws/src/robot\_gripper\_9\_91/robot\_gripper\_5\_29/robot\_gripper

包：arm\_description · arm\_moveit\_config · abb\_driver · control\_robot · robot\_serial · vi\_grab · robot\_ui · bio\_ik · 旁路包 vs\_practice

启动脚本：start.sh · start\_ui.sh · launch\_synarm\_ui.sh

不要：另起 ROS 2 课设仿真、改 abb\_driver、把 JEPA / 现有抓取 UI 和本练习混成一套

总顺序：

S 启动并下发运动 → A 登记接口 → B Commander 流程 → C 检测 → D 开环视觉 → E 闭环伺服 → F 对照表 → G 延迟三律 → H 真机细则

整条链要做成的事只有一句：先用规划把末端送到能看见标签的地方，然后停止规划长轨迹，每一帧用像素误差生成运动，臂一动图像就变，形成视觉—控制闭环。视觉只提供误差；思考在增益、限幅、延迟、停机。

每个新终端先执行：

```
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
cd "$WS"
```

## 模块 S 启动本机机械臂并下发运动指令

### 目的

证明执行层已经通了。后面所有视觉和控制，最后都要变成「臂按你的命令动一下」。如果现在还不会启动、不会在 RViz 里 Execute、不会用 Python 发 2 cm 位移，后面写检测器只是在空转。

本步要你能口头说清四件事：在哪一层目录启动；三条实验室脚本不要同时开；RViz Execute 是给 MoveIt 下目标；moveit\_commander.go() 和 RViz 是同一层客户端，不是 driver。

### 原理

本机运动指令只走这一条链：

```
你 (RViz 或 Python Commander)
→ MoveIt 1 (arm_moveit_config / move_group, IK 可能走 bio_ik)
→ FollowJointTrajectory
→ abb_driver / control_robot
→ 控制器或仿真
→ /joint_states、/tf 回来
```

start.sh 一类脚本负责把驱动和 MoveIt 拉起来。RViz 点 Execute，与 Python 里 set\_pose\_target 再 go()，都是「请规划器出一条关节轨迹，交给已经在跑的跟踪器」。abb\_driver 在最底层，本练习只调用、不改。

所以本步练的不是视觉，是：谁规划、谁跟踪、谁是驱动。分不清这三层，后面会把 go() 当成闭环，或把规划失败当成控制律失败。

### 与上一步的衔接

这是整条练习的起点。它衔接的不是上一个算法步骤，而是你已经具备的两样东西：

1. 实验室已经给了 Noetic 工作空间和启动脚本（你贴出的 listing）。
2. 课程第八章让你知道「会调用 Commander」，但那是 ROS 2 口吻；本机必须换成 MoveIt 1 的 moveit\_commander。

本步把「工作空间存在」变成「臂真的听命令」。没有这个，步骤 A 的接口清单没有活的话题可抄，步骤 B 的脚本没有 move\_group 可连。

## 怎么做

1. 每个终端按文首 source。不要对 /home/biand/workspace\_ws 跑 catkin build 或 colcon。
2. 先读脚本，不要猜：sed -n '1,80p' start.sh start\_ui.sh launch\_synarm\_ui.sh，再 ls src/arm\_moveit\_config/launch。
3. 只开一条 bringup。只练 MoveIt：roslaunch arm\_moveit\_config demo.launch。实验室日常：./start.sh。要 UI：./start\_ui.sh 或 ./launch\_synarm\_ui.sh。真机还要遵守组里使能 / ROS\_DOCUMENTATION.md。
4. 另开终端：rostopic echo /joint\_states -n 1 必须有数。
5. RViz → MotionPlanning → 选本机规划组 → 末端只拖 2~3 cm → Plan → 速度 0.1 → Execute。再 echo 一次关节，数字必须变。
6. 把本仓库 vs\_practice 拷进 \$WS/src/，catkin build vs\_practice，重新 source。然后：

```
roslaunch vs_practice send_small_motion.py _group:=manipulator _dz:=0.02 _dry_run:=true
roslaunch vs_practice send_small_motion.py _group:=manipulator _dz:=0.02 _dry_run:=false
```

\_group 必须改成实际组名。位移大于 4 cm 脚本会拒绝。通过标准：关节在刷新、Execute 成功、Python 打出 go ok=True。详细失败表见 START\_ARM.md。

交给下一步：一条已经在跑的 MoveIt + 驱动，以及一次成功的小位移经验。

## 步骤 A 环境确认与信息登记

---

### 目的

把本机名字写成一张清单，避免后面把 panda\_arm、/camera/image\_raw 抄进实验室包。本步不产生算法，只产生 vs\_practice/config/interface.yaml 和一份抄自本机的 waypoints.yaml 初值。

后面每个节点都要写死：规划组、末端 link、图像话题、servo\_mode。这些不是通用常数，是 arm\_moveit\_config、vi\_grab、control\_robot 里已经存在的字符串。

### 原理

ROS 1 节点靠话题名、参数名、MoveIt 规划组名互相找到对方。名字错了，节点能启动但订不到数据，或 Commander 报 Invalid group name。清单的作用是把「活系统上读到的事实」冻成配置，代码只读配置。

本机还要在清单里登记 伺服接口类型：ABB 经 abb\_driver 通常只有轨迹，没有课设那种关节速度控制器。这一项决定步骤 E 走 joint\_velocity 还是 cartesian\_increment。现在不查，步骤 E 会按 ROS 2 手册去切一个不存在的控制器。

### 与上一步的衔接

模块 S 证明：move\_group 在、关节在变、你能发运动。步骤 A 接着问：那个会动的系统，官方名字到底叫什么？

- S.5 / S.6 成功的末端位姿，就是 waypoints.yaml 的来源，不要用手册示例数字。
- S 里若已跑过 print\_interface.py，本步把打印结果写入 yaml，不必再猜。
- S 保持一条 bringup 不关；A 只是另开终端去读话题和 SRDF，不再启动第二套。

没有 S，A 抄到的是空话题列表；没有 A，B 的 arm\_group:= 只能瞎填。

### 怎么做

1. 臂保持模块 S 的启动，不要另开 start.sh。
2. 若还没拷包：cp -a vs\_practice \$WS/src/vs\_practice && cd \$WS && catkin build vs\_practice && source devel/setup.bash。
3. 执行：

```
roslaunch vs_practice print_interface.py
rostopic echo /joint_states -n 1
rostopic list | grep -E 'image|joint|traj|velocity|follow'
```

1. 打开 src/arm\_moveit\_config 的 SRDF 与 controllers.yaml，与打印结果对照。

2. 填 `interface.yaml`: `arm_group`、`ee_link`、`base_frame`、图像话题、`servo_mode` (先倾向 `cartesian_increment`, 除非明确看到速度话题)。
3. 把模块 S 成功的位姿写入 `waypoints.yaml`。

通过标准: 清单没有空关键项。没有 `controller_manager` 不算失败, 在 `yaml` 里注明即可。

交给下一步: 规划组名和一份本机试过的接近位姿, 步骤 B 才能写 `MoveGroupCommander`(组名)。

## 步骤 B 用 Commander 走完开环作业流程

---

### 目的

把模块 S 的「单次 2 cm」扩成可重复的状态机：APPROACH → DESCEND → ( 可选 GRASP ) → LIFT → DONE。以后视觉只替换「目标从哪来」，不重写接近逻辑。

本步要你能向自己复述：set\_pose\_target 只表达希望末端到哪；plan / go 搜索轨迹；execute 把 JointTrajectory 交给跟踪器。调用者不是驱动器。bio\_ik 若已写在 kinematics.yaml，规划时自动用，不要调它的 C++。

### 原理

开环作业的含义是：目标在进入规划前就固定了。执行过程中即使标签被挪走，轨迹仍按旧目标走完。这正是现有 vi\_grab「检测一次再规划」的结构，也是后面步骤 D 的基线、步骤 E 要打破的东西。

笛卡尔下降用 compute\_cartesian\_path：在笛卡尔空间插路点，再逆解成关节轨迹，仍由同一轨迹控制器跟踪。fraction < 0.95 表示直线插不完整，应减下降距离，而不是改 driver。

状态机的价值是：失败时停在哪一步是明确的。规划失败先改 waypoints.yaml ( 碰撞、超限、目标太远 )，不要改 MoveIt 或 abb\_driver。

### 与上一步的衔接

- 从 S 接：S.6 已经证明 go() 能驱动本机臂。B 复用同一个 API，只是连续调用多次，并加上下降段。
- 从 A 接：arm\_group、ee\_link、waypoints.yaml 必须来自 A 的清单。B 的 launch 写成 arm\_group:=<print\_interface 的组名>，禁止 panda\_arm。
- 交给 C 的缺口：B 全程不看图像。臂能走完流程，但不知道标签在不在画面里。所以下一步只加测量，不加控制。

若 A 的路点是抄手册的示例数，B 会在规划失败上卡住，看起来像 Commander 不会用，其实是目标不属于这台臂。

### 怎么做

1. 确认 waypoints.yaml 的数字来自本机 RViz 或 send\_small\_motion.py 打印的位姿。
2. 启动 ( 臂已在 S 的 bringup 上 )：

```
roslaunch vs_practice commander_only.launch arm_group:=<你的组名>
```

1. 看 /vs/commander\_state。APPROACH / DESCEND / DONE 或 ERROR 必须打出来。
2. 另开终端：roslaunch record /joint\_states /tf，连续自动跑两遍，中间不拖 RViz。

3. 无夹爪则省略 GRASP，在笔记写明。规划失败：先把接近点靠近当前位姿；笛卡尔失败：descend\_dz 减半。

通过标准：不手动点目标，脚本能走完全流程；你能指出代码里没有改 abb\_driver。

交给下一步：一条可重复的「盲走」作业骨架，以及开环运动的 bag。步骤 C 往这条骨架旁边挂眼睛。

## 步骤 C 眼在手相机与 ArUco 检测

---

### 目的

让图像成为系统的输入。本步只测量、不控制：看见标签为真，中心像素和相对图像中心的误差往外发。禁止把像素送给 MoveIt。

### 原理

针孔模型把空间点投到像素  $(u,v)$ 。ArUco 提供四个角点，中心取平均，作为本练习唯一的视觉特征。参考点取图像中心  $(cx,cy)$ （来自 camera\_info 的内参，没有内参时用宽高一半）。误差：

$$e = (u - cx, v - cy)$$

眼在手表示相机固连末端。法兰系和光学系不同：光学系通常 Z 沿光轴向前、X 向右、Y 向下。手眼  $T_{ee\_cam}$  是常值，应与 URDF / TF 一致。本步先把检测跑通；手眼不准主要伤害步骤 D 的一次位姿估计，对步骤 E 的像素对中比较宽容。

本机已有 vi\_grab 和 99-realsense-libusb.rules。再开第二套 RealSense 会抢设备、抢话题。正确做法是订阅已经存在的图像话题。

延迟队列也可以先做在检测节点里（image\_delay\_ms），但本步先保持 0。步骤 G 再打开。这样测量与控制律仍然分开。

### 与上一步的衔接

- B 不看图，C 只看图。两条线并行：B 继续能走盲接近；C 在旁边发布 /vs/tag\_ok、/vs/pixel、/vs/error。还没有节点把误差拿去动臂。
- A 提供图像话题名。rostopic list | grep -i image 的结果应写进 interface.yaml，C 的 launch 用这个名字，不猜 /camera/image\_raw。
- S / B 的 TF 树已经在。C 用 rosrn tf tf\_echo <ee\_link> <camera\_optical\_frame> 核对手眼，frame 名来自 A 的 ee\_link 和 RealSense / URDF。
- 交给 D 的东西：「看见」变成布尔量和像素。D 才会第一次用视觉去生成 Commander 的目标。

若 C 把像素直接 set\_pose\_target，你跳过了 D/E 的区分，项目会退化回 vi\_grab 的开环抓取。

### 怎么做

1. 臂和相机按实验室方式已启动（不要第二份 realsense2\_camera）。
2. rostopic list | grep -i image，把实际话题写入 interface.yaml。
3. 放置或打印 ArUco（DICT\_4X4\_50，边长约 0.05 m），保证在画面里。
4. 启动检测：

```
roslaunch vs_practice detect_only.launch image_topic:=<实际话题>
```

1. `rostopic echo /vs/tag_ok、/vs/error`; `image_view` 看 `/vs/debug_image` (框和十字)。
2. 遮挡标签, `tag_ok` 必须变假; 平移标签, 误差必须变。
3. `roslaunch tf_echo <ee_link> <camera_optical_frame>`。

通过标准: 看见为真、遮挡为假、像素随标签动。日志里没有 `go()` / `plan()`。

交给下一步: 稳定的 `/vs/tag_ok` 与像素。步骤 D 在 APPROACH 入口读一次。

## 步骤 D 开环视觉引导

### 目的

证明「看见」可以生成一个位姿目标，同时留下开环基线：图像只用一次，运动中途不更新。步骤 F 将拿这次的终值误差和步骤 E 比。

### 原理

若检测能给出相机系中标签位姿  $T_{cam\_tag}$ ，则

$$T_{base\_tag} = T_{base\_ee} * T_{ee\_cam} * T_{cam\_tag}$$

预对准位姿取标签前方、沿光轴后退一段（例如 0.20 m），姿态让相机对着标签。该式只在 APPROACH 开始时算一次，然后交给步骤 B 已经会的 `go()`。执行过程中标签再动，轨迹不变，故为开环。

若暂时只有像素、没有三维位姿：可用「写死深度 + 像素近似横向偏移」估一个粗目标。精度差，但足够当基线。步骤 E 不再依赖这次估的位姿，改为纯像素误差。

开环终值误差来自：手眼不准、规划公差、标签在运动中被挪走。闭环要压的就是这类残差。

### 与上一步的衔接

- C → D: C 保证 `/vs/tag_ok` 为真时才允许进入 APPROACH。D 不重新发明检测，只在入口取一次数。
- B → D: D 不写新的规划客户端，直接调用 B 的 Commander 流程。变的是目标来源：由 `waypoints.yaml` 的写死数，改成「视觉算一次」。
- A → D: `base_frame`、`ee_link`、手眼 frame 必须来自清单，连乘才不会串系。
- 和 E 的边界: D 到位后再读一次像素，记下来就停。不要在本步把误差送进速度环，否则 F 的「开环」样本不干净。

D 做完，你应能一句话：图像用了一次，所以是开环。说不清，说明还停在「能抓」而不是「能对比」。

### 怎么做

1. 检测节点保持 C 的启动；臂保持 S 的 `bringup`。
2. 启动：

```
roslaunch vs_practice vs_practice.launch mode:=open
rostopic pub /vs/start std_msgs/String "data: go"
```

1. 日志应出现：等待标签 → 一次算目标 → Commander APPROACH / DESCEND → DONE。运动中途不得再次 `plan()` 追标签。

2. 到位后读 `/vs/error`，记为开环终值。 `rosv bag record /joint_states /tf /vs/error /vs/state`。
3. 手动平移标签，再跑一遍，确认新目标变了；单次执行中途不重规划。

通过标准：能说明图像只用了一次；bag 里伺服使能保持假。

交给下一步：开环基线和一次视觉接近。步骤 E 在同一接近之后切断规划，改走误差。

## 步骤 E 静态视觉闭环（核心）

---

### 目的

停止规划长轨迹，让误差每个周期都变成运动。这是本项目与 vi\_grab / 步骤 D 的分界，也是进组演示的最低完成线。

做完应能书面写出：e 在哪个坐标系、运动命令在哪个坐标系、本机走的是速度接口还是短笛卡尔增量。

### 原理

误差仍是  $e = (u - cx, v - cy)$ 。简化图像律（深度用常值估计  $Z_{\text{hat}}$ ）：

$$v_x = -\lambda * Z_{\text{hat}} * e_u / f_x$$

$$v_y = -\lambda * Z_{\text{hat}} * e_v / f_y$$

符号必须用实机试：u 增大通常对应光学系 +X，映射到基座后可能要反号。先改符号，再加大  $\lambda$ 。

本机接口分两路（A 已登记）：

- 真有关节速度话题：servo\_mode: joint\_velocity,  $\dot{q} = J + V$ ，限幅后发布。这是教科书 IBVS。
- ABB / abb\_driver 常见只有轨迹：cartesian\_increment，每周期把  $(v_x, v_y)$  变成几毫米的笛卡尔增量，经 Commander 走短目标。频率大约 2 ~ 10 Hz， $\lambda$  必须更小。它仍是「误差 → 运动 → 新图像」，不是再 plan() 一条自由空间长轨迹。禁止用 2 Hz 的大位移点冒充闭环。

停机： $\|e\| < \varepsilon$  连续 N 周期 → HOLD，速度/增量清零。tag\_ok 连续丢失 M 帧 → ABORT。超速先饱和，仍超则 ABORT。看丢必须立刻停，这是安全，也是闭环还在用图像的证据。

调参顺序： $\lambda$  从刚能蠕动加到刚振荡，再回调 20% ~ 30%。先限幅，再加增益。只改参数，不改检测器。

### 与上一步的衔接

- D 是开环对照，E 是同一接近之后换执行律。状态机仍是：先 APPROACH（可沿用 D 或写死安全接近），成功后切 SERVO，此时禁止再调用长轨迹 plan()。
- C 继续提供 /vs/error 和 /vs/tag\_ok。E 不重新检测，只订阅。检测失败应表现为停机，而不是规划器去「找标签」。
- B 的 Commander 在 SERVO 段降级。APPROACH 还可以用 B；伺服段若走 cartesian\_increment，只用短增量，不再走 APPROACH → DESCEND 全流程。
- A 的 servo\_mode 在这里生效。没填接口就按 ROS 2 去 switch\_controllers，本机上会失败，看起来像控制律不会写。

D 与 E 必须能用同一套初始关节和同一张标签来比。这是步骤 F 的前提。

## 怎么做

1. 确认 interface.yaml 的 servo\_mode。不要假设存在 ros2\_control。
2. 启动：

```
roslaunch vs_practice vs_practice.launch mode:=closed  
rostopic pub /vs/start std_msgs/String "data: go"
```

1. 看 /vs/state: WAIT\_TAG → APPROACH → SERVO → HOLD。SERVO 段日志不得出现一次长轨迹规划。
2.  $\lambda$  从 0.2 左右加起。标签放在画面左、右、上各一次，看误差是否对中。
3. 手遮标签：/vs/ee\_twist 应变 0，臂停。
4. 反向运动：先改图像律符号或笛卡尔增量符号，再加  $\lambda$ 。

通过标准：伺服段零次长 plan()；误差收敛或阻尼收敛；遮挡后运动为 0。

交给下一步：一条可重复的闭环，以及调过的  $\lambda$ 。步骤 F 用它和 D 比数字。

## 步骤 F 开环 / 闭环对照

### 目的

把「稳、准、快」变成可重复数字，而不是视频观感。缺表视为本关未完成。本步不发明新算法，只规定公平比较和指标。

### 原理

开环（步骤 D）：一次视觉 + 一次规划。终值误差含标定与规划公差，过程中无反馈。

闭环（步骤 E）：过程中减误差。终值由  $\epsilon$ 、噪声、增益决定。 $\lambda$  大则快，但可能振荡。

公平比较要求同一初始关节、同一标签位置、同一相机、同一接近点。只改 `mode:=open|closed`。失败定义事先写死：超时、丢失、超限、碰撞。失败样本保留，不删。

指标从 bag 算，不目视填：

| 指标 | 计算                            |     |  |                         |
|----|-------------------------------|-----|--|-------------------------|
| 准  | 最后 0.5 s 的平均 $\epsilon$       | $e$ |  | $\epsilon$ （像素）         |
| 快  | 首次进入 $\epsilon$ 并保持 0.3 s 的时间 |     |  |                         |
| 稳  | $\epsilon$                    | $e$ |  | $\epsilon$ 过零或峰值次数；最大超调 |

结论必须能用控制语言说一句，例如「闭环更准，但  $\lambda=0.8$  时有一次振荡」。这是进组时别人会追问的部分。

### 与上一步的衔接

- D 提供开环样本生成器，E 提供闭环样本生成器。F 不改它们的内部，只规定重复次数和统计。
- C 的  $\epsilon$  vs error 是两边共用的测量。所以比的是控制，不是两个检测器。
- S / B 保证初始关节能被设回去。每次试验前回到同一关节，否则「闭环更快」可能只是起点更近。
- 交给 G 的缺口：F 的表是静止标签、无人延迟。G 要在这张表旁边加运动目标和延迟列。没有 F，G 的三律对比没有静态锚点。

### 怎么做

1. 参数 `mode:=open` 跑 D，到位后采 1 s 误差；`mode:=closed` 跑 E。各  $\geq 5$  次。
2. 每次：`rosbag record /joint_states /tf /vs/error /vs/state`，记下  $\lambda$ 、初始关节、是否失败。
3. 用 bag 填表，画  $|e|(t)$ 。禁止看视频填数。
4. 写一句结论。若闭环不优于开环：先查符号、 $\lambda$ 、是否其实还在长轨迹规划。

通过标准：有表、有曲线、有一句结论。

交给下一步：静态基线。步骤 G 只改 `image law()` 和延迟，不再换架构。

## 步骤 G 运动目标、延迟与三种控制律

### 目的

在步骤 E 已稳定的环上，加时变目标和过时反馈，迫使思考离散控制，而不是再做一个检测器。这是练习的含金量，也是以后写「延迟下的视觉伺服」增量的伏笔。

三种律必须共享测量、限幅、停机，只替换 `image_law()`。

### 原理

检测与控制之间插入延迟  $T_d$  后，律使用  $e(t - T_d)$ 。固定  $\lambda$  仍按「当前还差很多」加油门，相位滞后导致振荡。

- p (固定比例)：就是步骤 E 的律，延迟下最容易晃。作对照。
- gain\_schedule:  $\lambda(T_d) = \lambda_0 / (1 + T_d / \tau)$ 。减小开环增益，稳，可能变慢。
- predict:  $\hat{e} = e + \dot{e} T_d$  (差分加低通)。试图对齐相位；模型错则超调。

标签慢速往复 (1~2 cm/s) 时，开环 (D) 会完全跟不上，闭环才有跟踪误差可言。延迟用软件队列实现 (C 里已有 `image_delay_ms`)，不要用「人眼反应」冒充  $T_d$ 。

实验格：静止  $T_d=0$ ；静止  $T_d \in \{50, 100, 150\}$  ms；慢速运动 + 同一组延迟。每格重复多次。记录均值/峰值、振荡、失败率。写清预测在哪一档延迟变差。

### 与上一步的衔接

- E 提供唯一允许改的函数位置：vs\_practice 的 `image_law()`。G 不改检测器、不改 driver、不改状态机结构。
- F 的静止无延迟表是第 0 列。G 的每张新表都应能和 F 对上：同一  $\lambda_0$ 、同一  $\epsilon$ 、同一初始条件。
- C 的延迟队列是执行延迟的地方，不是在控制律里 sleep。控制律应以为自己拿到的就是「当前」误差；过时由队列制造。这样三种律面对的是同一过时测量。
- 还不到 H 的新算法。G 可以在仿真或实验室慢速挪标签完成。H 只是同一结构换真实延迟和手眼残差。

未完成 E、没有 F 的表，不得进入 G。否则「预测更好」没有对照。

### 怎么做

1. E 已稳定。标签沿直线慢速往复 (手挪或脚本改 pose)，先 1~2 cm/s。
2. vs\_detect 设 `image_delay_ms` 为 0 / 50 / 100 / 150。
3. vs\_servo 的 law 取 p / gain\_schedule / predict。只改这一处。
4. 按实验格重复，bag 与 F 相同的话题。

5. 出表：哪一档延迟下  $p$  振荡、降增益变稳、预测从哪一档变差。

通过标准：launch 能复现全部参数；能用「过时误差 + 增益」解释表格。

交给下一步：一套只改律、不改架构的实验方法。步骤 H 在真机延迟下复现 E，有时间再抽几格 G。

## 步骤 H 真机细则（本机已在实验室包上）

---

### 目的

在真实延迟、手眼误差、摩擦和限速下复现步骤 E（能做则带 F）。不更换算法主题。占机短，调参回记录本，不在臂上盲加  $\lambda$ 。

### 原理

真机与 demo/fake 的差异主要是：手眼不准、图像与关节不同步、关节有摩擦和厂家限速、急停规程。控制结构不变：接近仍调用 Commander，伺服仍走 A 登记的接口。实验室用现有 abb\_driver / control\_robot，不把任何课设 hardware 抄进来。

真机第一条运动仍然遵守模块 S： $\leq 4\text{ cm}$ ，速度 0.1，急停够得到。E 的  $\lambda$  从仿真可用值再砍一截开始。

### 与上一步的衔接

- S 已经是实验室启动方式。H 不是「另接一台臂」，而是：同一套 start.sh，把模块 S 的安全约束和步骤 E 的闭环放到真实使能上。
- A 的接口在真机上可能与 demo 不同。若 demo 是 fake 轨迹、真机是 abb\_driver 轨迹，servo\_mode 更应是 cartesian\_increment。H 允许改 yaml，不允许改驱动源码。
- E / F / G 的节点不变。变的是限幅、 $\lambda$ 、占机时间和急停。G 的延迟仍用软件，标签可用手缓慢挪。
- 和论文的衔接：练习到 E/F 可演示闭环。大三若投稿，增量应落在「真机 + 延迟稳定」或「6/7 轴对照」，而不是再换一套检测。

### 怎么做

1. 导师确认急停、使能、相机是否已在法兰上。按 ROS\_DOCUMENTATION.md 上电。
2. 复做模块 S 的 2 cm，确认真机 Execute 与 Python go()。
3. 复做 C：订阅现有图像，不新开相机进程。
4. 限速为更保守值，复现 E。有时间做 F 的少量次数。G 用手挪标签 + 软件延迟。
5. 全程 bag。结束松使能。 $\lambda$  只在记录本上加，下一轮再上机。

通过标准：真机能对中，或能解释失败来自标定/接口/限速，而不是逻辑抄错。占机保持短时、集中。

交给以后：一份可演示的真机闭环，以及 F/G 里能写进大三增量的数据习惯。

## 总衔接图（做完应能默画）

---

S 臂能听命令（RViz / Commander 2 cm）  
A 把组名、话题、伺服接口写成 yaml  
B 把单次 go 扩成盲走状态机 ← 开环作业骨架  
C 旁路挂上眼睛，只发 e 和 tag\_ok ← 还不控制  
D 入口用一次视觉驱动 B ← 开环基线  
E 接近后切断长规划，e 每周期驱动运动 ← 闭环（演示线）  
F 同一初始条件对比 D 与 E ← 数字  
G 只换 image\_law，加延迟与运动目标 ← 含金量  
H 同一结构，真机限速复现 E

每一步只补上一步缺的那一层。缺执行先做 S；缺名字先做 A；缺流程先做 B；缺眼睛先做 C；缺「用了一次图」先做 D；缺「每帧都用图」先做 E；缺数字先做 F；缺延迟思想先做 G。不要两层同时炸。